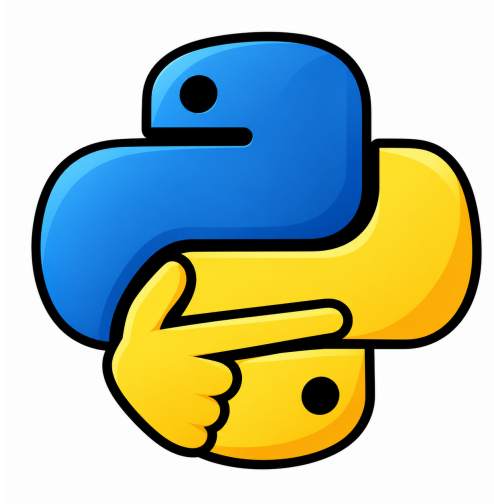




Python TRD for ICPC Competitions based on Pyrival

```
python_template.py . . . . . 1
.vimrc . . . . . 1
kactl_template.cpp . . . . . 1
kactl_troubleshoot.md . . . . . 1
algebra/discrete_log.py . . . . . 1
algebra/factors.py . . . . . 1
algebra/fft.py . . . . . 2
algebra/mod_sqrt.py . . . . . 2
algebra/modinv.py . . . . . 2
algebra/ntt.py . . . . . 2
algebra/phi.py . . . . . 3
combinatorics/nCr_mod.py . . . . . 3
data_structures/DisjointSetUnion.py . . . . . 3
data_structures/FenwickTree.py . . . . . 3
data_structures/LazySegmentTree.py . . . . . 3
data_structures/RangeMinimumQuery.py . . . . . 4
data_structures/SortedList.py . . . . . 4
data_structures/TwoSat.py . . . . . 5
geometry/convex_hull.py . . . . . 5
graphs/bellman_ford.py . . . . . 6
graphs/bfs.py . . . . . 6
graphs/dfs.py . . . . . 6
graphs/binary_lifting_on_tree.py . . . . . 6
graphs/bridges_and_articulation_points.md . . . . . 7
graphs/centroid_decomposition.py . . . . . 7
graphs/dijkstra.py . . . . . 7
graphs/dinic.py . . . . . 7
graphs/edge_bcc.py . . . . . 8
graphs/floyd_warshall.py . . . . . 8
graphs/heavy_light.py . . . . . 8
graphs/hopcroft_karp.py . . . . . 9
graphs/kruskal.py . . . . . 10
graphs/lca.py . . . . . 10
graphs/maximum_matching.py . . . . . 10
graphs/node_bcc.py . . . . . 11
graphs/prim.py . . . . . 11
graphs/rerooter.py . . . . . 11
graphs/scc.py . . . . . 12
graphs/toposort.py . . . . . 12
linear_algebra/matrix.py . . . . . 12
linear_algebra/max_xor.py . . . . . 13
misc/bit_hacks.py . . . . . 13
misc/bootstrap.py . . . . . 13
misc/mod.py . . . . . 13
numerical/berlekamp_massey.py . . . . . 13
numerical/integrate.py . . . . . 14
numerical/search.py . . . . . 14
strings/LPSubstr.py . . . . . 14
strings/hashing.py . . . . . 15
strings/kmp.py . . . . . 15
strings/lcs.py . . . . . 15
strings/min_rotation.py . . . . . 15
strings/suffix_array.py . . . . . 15
strings/z_algorithm.py . . . . . 16
```

python_template.py

```
import sys
# Avoid automatic flushes
input = lambda: sys.stdin.readline().rstrip()
```

.vimrc

```
set hlsearch
set number
set tabstop=4
```

```
set shiftwidth=4
set expandtab
```

kactl_template.cpp

```
#include <bits/stdc++.h>
using namespace std;

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;

int main() {
    cin.tie(0) -> sync_with_stdio(0);
    cin.exceptions(cin.failbit);
}
```

kactl_troubleshoot.md

Pre-submit: Write a few simple test cases if sample is not enough. Are time limits close? If so, generate max cases. Is the memory usage fine? Could anything overflow? Make sure to submit the right file.

Wrong answer: Print your solution! Print debug output, as well. Are you clearing all data structures between test cases? Can your algorithm handle the whole range of input? Read the full problem statement again. Do you handle all corner cases correctly? Have you understood the problem correctly? Any uninitialized variables? Any overflows? Confusing N and M, i and j, etc.? Are you sure your algorithm works? What special cases have you not thought of? Are you sure the STL functions you use work as you think? Add some assertions, maybe resubmit. Create some testcases to run your algorithm on. Go through the algorithm for a simple case. Go through this list again. Explain your algorithm to a teammate. Ask the teammate to look at your code. Go for a small walk, e.g. to the toilet. Is your output format correct? (including whitespace) Rewrite your solution from the start or let a teammate do it.

Runtime error: Have you tested all corner cases locally? Any uninitialized variables? Are you reading or writing outside the range of any vector? Any assertions that might fail? Any possible division by 0? (mod 0 for example) Any possible infinite recursion? Invalidated pointers or iterators? Are you using too much memory? Debug with resubmits (e.g. remapped signals, see Various).

Time limit exceeded: Do you have any possible infinite loops? What is the complexity of your algorithm? Are you copying a lot of unnecessary data? (References) How big is the input and output? (consider scanf) Avoid vector, map. (use arrays/unordered_map) What do your teammates think about your algorithm?

Memory limit exceeded: What is the max amount of memory your algorithm should need? Are you clearing all data structures between test cases?

algebra/discrete_log.py

```
def discrete_log(a, b, mod):
    """
    Returns smallest x > 0 s.t. pow(a, x, mod) == b or None
    if no such x exists.
    Note: works even if a and mod are not coprime.
    """
    n = int(mod**0.5) + 1

    # tiny_step[x] = maximum j <= n s.t. b * a^j % mod = x
    tiny_step, e = {}, 1
    for j in range(1, n + 1):
        e = e * a % mod
        if e == b:
            return j
        tiny_step[b * e % mod] = j

    # find (i, j) s.t. a^(n * i) % mod = b * a^j % mod
    factor = e
    for i in range(2, n + 2):
        e = e * factor % mod
        if e in tiny_step:
            j = tiny_step[e]
            return n * i - j if pow(a, n * i - j, mod) == b else None
```

algebra/factors.py

```
from collections import Counter
```

```
def gcd(x, y):
    """greatest common divisor of x and y"""
    while y:
        x, y = y, x % y
    return x

def memodict(f):
    """memoization decorator for a function taking a single
    argument"""
    class memodict(dict):
        def __missing__(self, key):
            ret = self[key] = f(key)
            return ret

    return memodict().__getitem__

def pollard_rho(n):
    """returns a random factor of n"""
    if n & 1 == 0:
        return 2
    if n % 3 == 0:
        return 3
```

```

s = ((n - 1) & (1 - n)).bit_length() - 1
d = n >> s
for a in [2, 325, 9375, 28178, 450775, 9780504, \
1795265022]:
    p = pow(a, d, n)
    if p == 1 or p == n - 1 or a % n == 0:
        continue
    for _ in range(s):
        prev = p
        p = (p * p) % n
        if p == 1:
            return gcd(prev - 1, n)
        if p == n - 1:
            break
    else:
        for i in range(2, n):
            x, y = i, (i * i + i) % n
            f = gcd(abs(x - y), n)
            while f == 1:
                x, y = (x * x + i) % n, (y * y + i) % n
                y = (y * y + i) % n
                f = gcd(abs(x - y), n)
            if f != n:
                return f
    return n

```

```

@memodict
def prime_factors(n):
    """returns a Counter of the prime factorization of n"""
    if n <= 1:
        return Counter()
    f = pollard_rho(n)
    return Counter([n]) if f == n else prime_factors(f) + \
prime_factors(n // f)

```

```

def distinct_factors(n):
    """returns a list of all distinct factors of n"""
    factors = [1]
    for p, exp in prime_factors(n).items():
        factors += [p**i * factor for factor in factors for i \
in range(1, exp + 1)]
    return factors

```

```

def all_factors(n):
    """returns a sorted list of all distinct factors of n"""
    small, large = [], []
    for i in range(1, int(n**0.5) + 1, 2 if n & 1 else 1):
        if not n % i:
            small.append(i)
            large.append(n // i)
    if small[-1] == large[-1]:
        large.pop()
    large.reverse()
    small.extend(large)
    return small

```

algebra/fft.py

```

# FFT implementation based on \
https://codeforces.com/blog/entry/117947

rt = [1]
def fft(P):
    n = len(P)
    P = list(P)
    assert n and (n - 1) & n == 0

    while 2 * len(rt) < n:
        import cmath
        root = cmath.exp(2j * cmath.pi / (4 * len(rt)))
        rt.extend([r * root for r in rt])

    k = n
    while k > 1:
        for i in range(n//k):
            r = rt[i]
            for j1 in range(i*k, i*k + k//2):
                j2 = j1 + k//2
                z = r * P[j2]
                P[j2] = P[j1] - z
                P[j1] = P[j1] + z
            k //= 2

    rev = [0] * n
    for i in range(1, n):
        rev[i] = rev[i // 2] // 2 + (i & 1) * n // 2
    return [P[r] for r in rev]

```

```

def ifft(P):
    n = len(P)
    return fft([P[-i]/n for i in range(n)])

```

```

def fft_conv(P, Q):
    m = len(P) + len(Q) - 1
    n = 1 << m.bit_length()

    P = P + [0] * (n - len(P))
    Q = Q + [0] * (n - len(Q))
    P, Q = fft(P), fft(Q)

    return ifft([p*q for p,q in zip(P, Q)])[:m]

```

algebra/mod_sqrt.py

```

def mod_sqrt(a, p):
    """returns x s.t. x**2 == a (mod p)"""
    a %= p
    if a < 2:
        return a
    assert pow(a, (p - 1) // 2, p) == 1
    if p & 3 == 3:
        return pow(a, (p + 1) // 4, p)

    r = ((p - 1) & (1 - p)).bit_length() - 1
    s, n = p >> r, 2
    while pow(n, (p - 1) // 2, p) != p - 1:

```

```

        n += 1

    x, b, g = pow(a, (s + 1) // 2, p), pow(a, s, p), pow(n, \
s, p)
    while True:
        t = b
        for m in range(r):
            if t == 1:
                break
            t = (t * t) % p
        if m == 0:
            return x

    gs = pow(g, 1 << (r - m - 1), p)
    g, x = (gs * gs) % p, (x * gs) % p
    b, r = (b * g) % p, m

```

algebra/modinv.py

```

def extended_gcd(a, b):
    """returns gcd(a, b), s, r s.t. a * s + b * r == gcd(a, \
b)"""
    s, old_s = 0, 1
    r, old_r = b, a
    while r:
        q = old_r // r
        old_r, r = r, old_r - q * r
        old_s, s = s, old_s - q * s
    return old_r, old_s, (old_r - old_s * a) // b if b else 0

```

```

def modinv(a, m):
    """returns the modular inverse of a w.r.t. to m, works \
when a and m are coprime"""
    g, x, _ = extended_gcd(a % m, m)
    return x % m if g == 1 else None

```

algebra/ntt.py

```

# NTT implementation based on \
https://codeforces.com/blog/entry/117947

```

```

# NTT prime
MOD = (119 << 23) + 1
assert MOD & 1

```

```

non_quad_res = 2
while pow(non_quad_res, MOD//2, MOD) != MOD - 1:
    non_quad_res += 1
rt = [1]

```

```

def ntt(P):
    n = len(P)
    P = list(P)
    assert n and (n - 1) & n == 0

```

```

    while 2 * len(rt) < n:
        # 4*len(rt)-th root of unity
        root = pow(non_quad_res, MOD // (4*len(rt)), MOD)
        rt.extend([r * root % MOD for r in rt])

```

```

k = n
while k > 1:
    for i in range(n//k):
        r = rt[i]
        for j1 in range(i*k, i*k + k//2):
            j2 = j1 + k//2
            z = r * P[j2]
            P[j2] = (P[j1] - z) % MOD
            P[j1] = (P[j1] + z) % MOD
        k //= 2

rev = [0] * n
for i in range(1, n):
    rev[i] = rev[i // 2] // 2 + (i & 1) * n // 2
return [P[r] for r in rev]

def intt(P):
    n = len(P)
    ninv = pow(n, MOD - 2, MOD)
    return ntt([P[-i] * ninv % MOD for i in range(n)])

def ntt_conv(P, Q):
    m = len(P) + len(Q) - 1
    n = 1 << m.bit_length()

    P = P + [0] * (n - len(P))
    Q = Q + [0] * (n - len(Q))
    P, Q = ntt(P), ntt(Q)

    return intt([p * q % MOD for p, q in zip(P, Q)]):m]

```

algebra/phi.py

```

def phi(n):
    """returns phi(x) for all x <= n"""
    sieve = [i if i & 1 else i // 2 for i in range(n + 1)]
    for i in range(3, n + 1, 2):
        if sieve[i] == i:
            for j in range(i, n + 1, i):
                sieve[j] = (sieve[j] // i) * (i - 1)

    return sieve

```

combinatorics/nCr_mod.py

```

def make_nCr_mod(max_n=2 * 10**5, mod=10**9 + 7):
    max_n = min(max_n, mod - 1)

    fact, inv_fact = [0] * (max_n + 1), [0] * (max_n + 1)
    fact[0] = 1
    for i in range(max_n):
        fact[i + 1] = fact[i] * (i + 1) % mod

    inv_fact[-1] = pow(fact[-1], mod - 2, mod)
    for i in reversed(range(max_n)):
        inv_fact[i] = inv_fact[i + 1] * (i + 1) % mod

    def nCr_mod(n, r):
        res = 1

```

```

while n or r:
    a, b = n % mod, r % mod
    if a < b:
        return 0
    res = res * fact[a] % mod * inv_fact[b] % mod * \
    inv_fact[a - b] % mod
    n //= mod
    r //= mod
    return res

return nCr_mod

```

data_structures/DisjointSetUnion.py

```

class DisjointSetUnion:
    def __init__(self, n):
        self.parent = list(range(n))
        self.size = [1] * n
        self.num_sets = n

    def find(self, a):
        acopy = a
        while a != self.parent[a]:
            a = self.parent[a]
        while acopy != a:
            self.parent[acopy], acopy = a, self.parent[acopy]
        return a

    def union(self, a, b):
        a, b = self.find(a), self.find(b)
        if a != b:
            if self.size[a] < self.size[b]:
                a, b = b, a

            self.num_sets -= 1
            self.parent[b] = a
            self.size[a] += self.size[b]

    def set_size(self, a):
        return self.size[self.find(a)]

    def __len__(self):
        return self.num_sets

```

class UnionFind:

```

    def __init__(self, n):
        self.parent = list(range(n))

    def find(self, a):
        acopy = a
        while a != self.parent[a]:
            a = self.parent[a]
        while acopy != a:
            self.parent[acopy], acopy = a, self.parent[acopy]
        return a

    def union(self, a, b):
        self.parent[self.find(b)] = self.find(a)

```

data_structures/FenwickTree.py

```

class FenwickTree:
    def __init__(self, x):
        """transform list into BIT"""
        self.bit = x
        for i in range(len(x)):
            j = i | (i + 1)
            if j < len(x):
                x[j] += x[i]

    def update(self, idx, x):
        """updates bit[idx] += x"""
        while idx < len(self.bit):
            self.bit[idx] += x
            idx |= idx + 1

    def query(self, end):
        """calc sum(bit[:end])"""
        x = 0
        while end:
            x += self.bit[end - 1]
            end &= end - 1
        return x

    def findkth(self, k):
        """Find largest idx such that sum(bit[:idx]) <= k"""
        idx = -1
        for d in reversed(range(len(self.bit).bit_length())):
            right_idx = idx + (1 << d)
            if right_idx < len(self.bit) and k >= \
self.bit[right_idx]:
                idx = right_idx
                k -= self.bit[idx]
        return idx + 1

```

data_structures/LazySegmentTree.py

```

class LazySegmentTree:
    """Lazy segment tree with source-editable commutative \
updates and aggregates."""

    def __init__(self, data, default=0, func=max):
        """initialize the lazy segment tree with data"""
        self._default = default
        self._func = func

        self._len = len(data)
        self._size = 1 << (self._len - 1).bit_length()
        self._buffer_idx = 2 * _size

        # For custom operations, replace this state and the \
four methods below.
        # Multi-field state also requires changing the marked \
buffer operations.
        self._lazy = [0] * (2 * _size + 1)
        self.data = [default] * (2 * _size + 1)
        self.data[_size:_size + self._len] = data
        for i in range(_size - 1, 0, -1):
            self._merge_data(i + i, i + i + 1, i)

```

```

def __len__(self):
    return self._len

def _get_range(self, idx):
    """return the leaf range represented by idx"""
    shift = self._size.bit_length() - idx.bit_length()
    return idx << shift, (idx + 1) << shift

def _unset_lazy(self, idx):
    """clear the lazy update at idx"""
    self._lazy[idx] = 0

def _apply_to_data(self, update_idx, data_idx):
    """apply the lazy update at update_idx to data_idx"""
    self.data[data_idx] += self._lazy[update_idx]

def _apply_to_lazy(self, update_idx, lazy_idx):
    """compose the update at update_idx into lazy_idx"""
    self._lazy[lazy_idx] += self._lazy[update_idx]

def _merge_data(self, left, right, target):
    """merge the data at left and right into target"""
    self.data[target] = self._func(self.data[left], \
self.data[right])

def _update(self, idx):
    """apply all updates stored above idx"""
    for i in range(idx.bit_length() - 1, 0, -1):
        parent = idx >> i
        left = parent + parent
        self._apply_to_data(parent, left)
        self._apply_to_lazy(parent, left)
        self._apply_to_data(parent, left + 1)
        self._apply_to_lazy(parent, left + 1)
        self._unset_lazy(parent)

def _build(self, idx):
    """make the changes to idx known to its ancestors"""
    idx >>= 1
    while idx:
        self._merge_data(idx + idx, idx + idx + 1, idx)
        self._apply_to_data(idx, idx)
        idx >>= 1

def apply(self, start, stop, value):
    """lazily apply value to [start, stop)"""
    start = start_copy = start + self._size
    stop = stop_copy = stop + self._size
    buffer_idx = self._buffer_idx

    # For multi-field updates, initialize every lazy \
buffer field here.
    self._lazy[buffer_idx] = value

    while start < stop:
        if start & 1:
            self._apply_to_lazy(buffer_idx, start)
            self._apply_to_data(buffer_idx, start)

```

```

            start += 1
        if stop & 1:
            stop -= 1
            self._apply_to_lazy(buffer_idx, stop)
            self._apply_to_data(buffer_idx, stop)
        start >>= 1
        stop >>= 1

    # Tell all nodes above the updated area of the updates.
    self._build(start_copy)
    self._build(stop_copy - 1)

add = apply

def query(self, start, stop, default=0):
    """return func(data[start:stop])"""
    start += self._size
    stop += self._size

    # Apply all the lazily stored updates.
    self._update(start)
    self._update(stop - 1)

    buffer_idx = self._buffer_idx

    # For multi-field data, initialize every data buffer \
field here.
    self.data[buffer_idx] = default
    while start < stop:
        if start & 1:
            self._merge_data(buffer_idx, start, buffer_idx)
            start += 1
        if stop & 1:
            stop -= 1
            self._merge_data(buffer_idx, stop, buffer_idx)
            start >>= 1
            stop >>= 1

    # For multi-field data, return every data buffer field \
here.
    return self.data[buffer_idx]

def __repr__(self):
    return "LazySegmentTree({0})".format(self.data)

```

data_structures/RangeMinimumQuery.py

```

"""
Range Minimum Query (RMQ), also known as a sparse table.
Precomputation takes O(n log n) time and memory.
Query takes O(1) time.

To switch to a max version, simply search replace min to \
max.
"""
class RangeMinimumQuery:
    def __init__(self, data):
        self._data = [data]
        n = len(data)
        for bit in range(n.bit_length() - 1):

```

```

            i = 1 << bit
            data = [min(data[j], data[j + i]) for j in range(n - \
2 * i + 1)]
            self._data.append(data)

def query(self, start, stop):
    """min of data[start, stop)"""
    depth = (stop - start).bit_length() - 1
    data = self._data[depth]
    return min(data[start], data[stop - (1 << depth)])

"""
"Improved" Range Minimum Query (RMQ) based on \
https://codeforces.com/blog/entry/78931
Uses the above RMQ as a subroutine.

Precomputation takes O(n) time and memory.
Query takes O(1) time (but with worse constant factor than \
above RMQ!).

To switch to a max version, simply search replace min to \
max in both RMQs.
"""
class RangeMinimumQuery2:
    def __init__(self, data):
        self.data = data
        n = len(data)
        mask = 0
        self.A = A = [0] * n
        for i in range(n)[::-1]:
            while mask and min(data[i + (mask & \
-mask).bit_length()], data[i]) == data[i]:
                mask &= mask - 1
                A[i] = mask = 2 * (mask & ~(1 << 62)) + 1
            self.B = [data[i + (A[i] & ~(1 << 32)).bit_length() - \
1] for i in range(n)]
            self._query = RangeMinimumQuery(self.B[:32]).query

    def query(self, start, stop):
        """min of data[start, stop)"""
        if stop - start < 64:
            return self.data[start + (self.A[start] & ~(1 << \
stop - start)).bit_length() - 1]
        return min(self.B[start], self.B[stop - 32], \
self._query((start + 31) >> 5, stop >> 5))

```

data_structures/SortedList.py

```

"""
The "sorted list" data-structure, with amortized \
O(n^(1/3)) cost per insert and pop.

Example:

A = SortedList()
A.insert(30)
A.insert(50)
A.insert(20)

```

```

A.insert(30)
A.insert(30)

print(A) # prints [20, 30, 30, 30, 50]

print(A.lower_bound(30), A.upper_bound(30)) # prints 1 4

print(A[-1]) # prints 50
print(A.pop(1)) # prints 30

print(A) # prints [20, 30, 30, 50]
print(A.count(30)) # prints 2

"""

from bisect import bisect_left as lower_bound
from bisect import bisect_right as upper_bound

class FenwickTree:
    def __init__(self, x):
        bit = self.bit = list(x)
        size = self.size = len(bit)
        for i in range(size):
            j = i | (i + 1)
            if j < size:
                bit[j] += bit[i]

    def update(self, idx, x):
        """updates bit[idx] += x"""
        while idx < self.size:
            self.bit[idx] += x
            idx |= idx + 1

    def __call__(self, end):
        """calc sum(bit[:end])"""
        x = 0
        while end:
            x += self.bit[end - 1]
            end &= end - 1
        return x

    def find_kth(self, k):
        """Find largest idx such that sum(bit[:idx]) <= k"""
        idx = -1
        for d in reversed(range(self.size.bit_length())):
            right_idx = idx + (1 << d)
            if right_idx < self.size and self.bit[right_idx] <= k:
                idx = right_idx
                k -= self.bit[idx]
        return idx + 1, k

class SortedList:
    block_size = 700

    def __init__(self, iterable=()):
        iterable = sorted(iterable)
        self.micros = [iterable[i:i + self.block_size - 1] for \

```

```

i in range(0, len(iterable), self.block_size - 1)] or [[]]
        self.macro = [i[0] for i in self.micros[1:]]
        self.micro_size = [len(i) for i in self.micros]
        self.fenwick = FenwickTree(self.micro_size)
        self.size = len(iterable)

    def insert(self, x):
        i = lower_bound(self.macro, x)
        j = upper_bound(self.micros[i], x)
        self.micros[i].insert(j, x)
        self.size += 1
        self.micro_size[i] += 1
        self.fenwick.update(i, 1)
        if len(self.micros[i]) >= self.block_size:
            self.micros[i:i + 1] = \
self.micros[i][:self.block_size >> 1], \
self.micros[i][self.block_size >> 1:]
            self.micro_size[i:i + 1] = self.block_size >> 1, \
self.block_size >> 1
            self.fenwick = FenwickTree(self.micro_size)
            self.macro.insert(i, self.micros[i + 1][0])

    def pop(self, k=-1):
        i, j = self._find_kth(k)
        self.size -= 1
        self.micro_size[i] -= 1
        self.fenwick.update(i, -1)
        return self.micros[i].pop(j)

    def __getitem__(self, k):
        i, j = self._find_kth(k)
        return self.micros[i][j]

    def count(self, x):
        return self.upper_bound(x) - self.lower_bound(x)

    def __contains__(self, x):
        return self.count(x) > 0

    def lower_bound(self, x):
        i = lower_bound(self.macro, x)
        return self.fenwick(i) + lower_bound(self.micros[i], x)

    def upper_bound(self, x):
        i = upper_bound(self.macro, x)
        return self.fenwick(i) + upper_bound(self.micros[i], x)

    def _find_kth(self, k):
        return self.fenwick.find_kth(k + self.size if k < 0 \
else k)

    def __len__(self):
        return self.size

    def __iter__(self):
        return (x for micro in self.micros for x in micro)

    def __repr__(self):
        return str(list(self))

```

data_structures/TwoSat.py

```

def find_SCC(graph):
    SCC, S, P = [], [], []
    depth = [0] * len(graph)

    stack = list(range(len(graph)))
    while stack:
        node = stack.pop()
        if node < 0:
            d = depth[~node] - 1
            if P[-1] > d:
                SCC.append(S[d:])
                del S[d:], P[-1]
                for node in SCC[-1]:
                    depth[node] = -1
            elif depth[node] > 0:
                while P[-1] > depth[node]:
                    P.pop()
            elif depth[node] == 0:
                S.append(node)
                P.append(len(S))
                depth[node] = len(S)
                stack.append(~node)
                stack += graph[node]
    return SCC[::-1]

class TwoSat:
    def __init__(self, n):
        self.n = n
        self.graph = [[] for _ in range(2 * n)]

    def _imply(self, x, y):
        self.graph[x].append(y if y >= 0 else 2 * self.n + y)

    def either(self, x, y):
        """either x or y must be True"""
        self._imply(~x, y)
        self._imply(~y, x)

    def set(self, x):
        """x must be True"""
        self._imply(~x, x)

    def solve(self):
        SCC = find_SCC(self.graph)
        order = [0] * (2 * self.n)
        for i, comp in enumerate(SCC):
            for x in comp:
                order[x] = i
            for i in range(self.n):
                if order[i] == order[~i]:
                    return False, None
            return True, [(order[i] > order[~i]) for i in \
range(self.n)]

```

geometry/convex_hull.py

```

"""
Given a list P of n points in 2D, convex_hull computes \

```

its convex hull in $O(n \log n)$ time.

The points are returned clock-wise starting with the \ left-most upper-most point (i.e. $\min(P)$).

```
def convex_hull(P):
    P = sorted(P)
    cross = lambda a,b,c: (a[0] - b[0]) * (c[1] - b[1]) - \
(a[1] - b[1]) * (c[0] - b[0])
    H = []
    for p in P + P[::-1]:
        while len(H) > 1 and cross(H[-2], H[-1], p) <= 0 and \
not H[-2] < H[-1] > p:
            H.pop()
        H.append(p)
    return H[:-1]
```

graphs/bellman_ford.py

```
def bellman_ford(n, edges, start):
    dist = [float("inf")] * n
    pred = [None] * n

    dist[start] = 0

    for _ in range(n):
        for u, v, d in edges:
            if dist[u] + d < dist[v]:
                dist[v] = dist[u] + d
                pred[v] = u

    """Sanity Check
    for u, v, d in edges:
        if dist[u] + d < dist[v]:
            return None
    """

    return dist, pred
```

graphs/bfs.py

```
def bfs(graph, start=0):
    used = [False] * len(graph)
    used[start] = True
    q = [start]
    for v in q:
        for w in graph[v]:
            if not used[w]:
                used[w] = True
                q.append(w)
    return q

def layers(graph, start=0):
    used = [False] * len(graph)
    used[start] = True
    q, ret = [start], []
    while q:
        nq = []
        ret.append(q)
        for v in q:
```

```
for w in graph[v]:
    if not used[w]:
        used[w] = True
        nq.append(w)
    q = nq
    return ret
```

graphs/dfs.py

```
def dfs(graph, start=0):
    n = len(graph)

    dp = [0] * n
    visited, finished = [False] * n, [False] * n

    stack = [start]
    while stack:
        start = stack[-1]

        # push unvisited children into stack
        if not visited[start]:
            visited[start] = True
            for child in graph[start]:
                if not visited[child]:
                    stack.append(child)
        else:
            stack.pop()

            # base case
            dp[start] += 1

            # update with finished children
            for child in graph[start]:
                if finished[child]:
                    dp[start] += dp[child]

            finished[start] = True

    return visited, dp
```

graphs/binary_lifting_on_tree.py

Binary lifting technique applied to a tree.
There are four different uses of this implementation

1. Computing LCA in $O(\log n)$ time.

```
Example:
    0
    |
    1
   /\
  2 3

graph = [[1], [0, 2, 3], [1], [1]]
BL = binary_lift(graph, root=0)
print(BL.lca(2, 3)) # prints 1
```

2. Compute the k-th ancestor of a node in $O(\log k)$ time.

Example:

```
graph = [[1], [0, 2, 3], [1], [1]]
BL = binary_lift(graph, root=0)
print(BL.kth_ancestor(2, 2)) # prints 0
print(BL.kth_ancestor(2, 3)) # prints -1
```

3. Compute the distance between two nodes in $O(\log n)$ time.

Example:

```
graph = [[1], [0, 2, 3], [1], [1]]
BL = binary_lift(graph)
print(BL.distance(2, 3)) # prints 2
```

4. Compute the sum/min/max/... of the weight of a path between a pair of nodes in $O(\log n)$ time.

```
res = data[Path[0]]
for node in Path[1:]:
    res = f(res, data[node])
return res
```

Example:

```
graph = [[1], [0, 2, 3], [1], [1]]
data = [1, 10, 20, 5]
BL = binary_lift(graph, data, f = lambda a,b: a + b)
print(BL(2, 3)) # prints 35
```

class binary_lift:

```
def __init__(self, graph, data=(), f=min, root=0):
    n = len(graph)

    parent = [-1] * (n + 1)
    depth = self.depth = [-1] * n
    bfs = [root]
    depth[root] = 0
    for node in bfs:
        for nei in graph[node]:
            if depth[nei] == -1:
                parent[nei] = node
                depth[nei] = depth[node] + 1
                bfs.append(nei)

    data = self.data = [data]
    parent = self.parent = [parent]
    self.f = f

    for _ in range(max(depth).bit_length()):
        old_data = data[-1]
        old_parent = parent[-1]

        data.append([f(val, old_data[p]) for val,p in \
zip(old_data, old_parent)])
        parent.append([old_parent[p] for p in old_parent])

    def lca(self, a, b):
        depth = self.depth
```

```

parent = self.parent

if depth[a] < depth[b]:
    a,b = b,a

d = depth[a] - depth[b]
for i in range(d.bit_length()):
    if (d >> i) & 1:
        a = parent[i][a]

for i in range(depth[a].bit_length()[::-1]):
    if parent[i][a] != parent[i][b]:
        a = parent[i][a]
        b = parent[i][b]

if a != b:
    return parent[0][a]
else:
    return a

def distance(self, a, b):
    return self.depth[a] + self.depth[b] - 2 * \
self.depth[self.lca(a,b)]

def kth_ancestor(self, a, k):
    parent = self.parent
    if self.depth[a] < k:
        return -1
    for i in range(k.bit_length()):
        if (k >> i) & 1:
            a = parent[i][a]
    return a

def __call__(self, a, b):
    depth = self.depth
    parent = self.parent
    data = self.data
    f = self.f

    c = self.lca(a, b)
    val = data[0][c]
    for x,d in (a, depth[a] - depth[c]), (b, depth[b] - \
depth[c]):
        for i in range(d.bit_length()):
            if (d >> i) & 1:
                val = f(val, data[i][x])
                x = parent[i][x]

    return val

```

graphs/bridges_and_articulation_points.md

This file exists to clarify that pyrrival does support \
bridges and articulation points.

* If you want to find the bridges of an undirected graph, \
then see 'edge_bcc.py'.

* If you want to find the articulation points of an \
undirected graph, then see 'node_bcc.py'.

graphs/centroid_decomposition.py

```

def centroid_decomposition(graph):
    """
    Given a tree, this function is a generator that
    1. Roots the tree at its centroid (modifying graph)
    2. Yields centroid
    3. Removes centroid from the graph
    4. Recurses on the forest left after the removal

    This generator makes working with centroid decomposition \
easy. It yields the n
    centroids involved in the decomposition. It also keeps \
the graph rooted at the yielded
    centroid by modifying the input variable graph. In total \
this takes O(n log n) time.

    Input:
    graph: list of lists where graph[u] is a list \
containing all neighbours of node u

    Example:
    >>> graph = [[1], [0,2], [1]]
    >>> for centroid in centroid_decomposition(graph):
    >>>     bfs = [centroid]
    >>>     for node in bfs:
    >>>         bfs += graph[node] # Valid since graph is rooted \
at the centroid
    >>>     print('BFS from centroid:', bfs)
    BFS from centroid: [1, 0, 2]
    BFS from centroid: [0]
    BFS from centroid: [2]
    """
    n = len(graph)

    bfs = [n - 1]
    for node in bfs:
        bfs += graph[node]
        for nei in graph[node]:
            graph[nei].remove(node)

    size = [0] * n
    for node in reversed(bfs):
        size[node] = 1 + sum(size[child] for child in \
graph[node])

    def reroot_centroid(root):
        N = size[root]
        while True:
            for child in graph[root]:
                if size[child] > N // 2:
                    size[root] = N - size[child]
                    graph[root].remove(child)
                    graph[child].append(root)
                    root = child
                    break
            else:
                return root

    bfs = [n - 1]

```

```

for node in bfs:
    centroid = reroot_centroid(node)
    bfs += graph[centroid]
    yield centroid

```

graphs/dijkstra.py

```
from heapq import heappop, heappush
```

```

def dijkstra(graph, start):
    """
    Uses Dijkstra's algorithm to find the shortest path \
from node start
    to all other nodes in a directed weighted graph.
    """
    n = len(graph)
    dist, parents = [float("inf")] * n, [-1] * n
    dist[start] = 0

    queue = [(0, start)]
    while queue:
        path_len, v = heappop(queue)
        if path_len == dist[v]:
            for w, edge_len in graph[v]:
                if edge_len + path_len < dist[w]:
                    dist[w], parents[w] = edge_len + path_len, v
                    heappush(queue, (edge_len + path_len, w))

    return dist, parents

```

graphs/dinic.py

```

INF = float("inf")

class Dinic:
    def __init__(self, n):
        self.lvl = [0] * n
        self.ptr = [0] * n
        self.q = [0] * n
        self.adj = [[] for _ in range(n)]

    def add_edge(self, a, b, c, rcap=0):
        self.adj[a].append([b, len(self.adj[b]), c, 0])
        self.adj[b].append([a, len(self.adj[a]) - 1, rcap, 0])

    def dfs(self, v, t, f):
        if v == t or not f:
            return f

        for i in range(self.ptr[v], len(self.adj[v])):
            e = self.adj[v][i]
            if self.lvl[e[0]] == self.lvl[v] + 1:
                p = self.dfs(e[0], t, min(f, e[2] - e[3]))
                if p:
                    self.adj[v][i][3] += p
                    self.adj[e[0]][e[1]][3] -= p
                    return p
            self.ptr[v] += 1

```

```

    return 0

def calc(self, s, t):
    flow, self.q[0] = 0, s
    for l in range(31): # l = 30 maybe faster for random \
data
        while True:
            self.lvl, self.ptr = [0] * len(self.q), [0] * \
len(self.q)
            qi, qe, self.lvl[s] = 0, 1, 1
            while qi < qe and not self.lvl[t]:
                v = self.q[qi]
                qi += 1
                for e in self.adj[v]:
                    if not self.lvl[e[0]] and (e[2] - e[3]) >> (30 \
- 1):
                        self.q[qe] = e[0]
                        qe += 1
                        self.lvl[e[0]] = self.lvl[v] + 1

            p = self.dfs(s, t, INF)
            while p:
                flow += p
                p = self.dfs(s, t, INF)

            if not self.lvl[t]:
                break

    return flow

```

graphs/edge_bcc.py

```

"""
REMARK: There are two definitions of biconnectedness of an \
undirected graph,
    either node-bcc or edge-bcc, corresponding to \
node_bcc.py and edge_bcc.py.

* In node-bcc, a "cut-vertex" (also called an articulation \
point) is a node where
    removing it would split up the graph into multiple \
connected components.

* In edge-bcc, a "bridge" is an edge where removing it \
would split up the graph
    into multiple connected components.

For example, this graph

    1  4  7  10
    / \ / \ / \
2  0  5  9
  \ / \ / \ /
    3  6  8

has three cut vertices (nodes 0,5,9) and one bridge (edge \
between 9 and 10).
The number of node-bcc = 4 and number of edge-bcc = 2.
Note that a node belongs to multiple node-bccs iff it is a \

```

```

cut-vertex.
"""

# Edge-bcc uses pyrrival's scc.py as a subroutine
"""
Given a directed graph, find_SCC returns a list of lists \
containing
the strongly connected components in topological order.

Note that this implementation can be also be used to check \
if a directed graph is a
DAG, and in that case it can be used to find the \
topological ordering of the nodes.
"""

def find_SCC(graph):
    SCC, S, P = [], [], []
    depth = [0] * len(graph)

    stack = list(range(len(graph)))
    while stack:
        node = stack.pop()
        if node < 0:
            d = depth[~node] - 1
            if P[-1] > d:
                SCC.append(S[d:])
                del S[d:], P[-1]
                for node in SCC[-1]:
                    depth[node] = -1
            elif depth[node] > 0:
                while P[-1] > depth[node]:
                    P.pop()
            elif depth[node] == 0:
                S.append(node)
                P.append(len(S))
                depth[node] = len(S)
                stack.append(~node)
                stack += graph[node]
    return SCC[::-1]

```

```

"""
Given an undirected simple graph, find_BCC returns a list \
of lists
containing the edge biconnected components of the graph \
(i.e. no bridges).
In a biconnected component, any two nodes are connected by \
at least two
edge-disjoint paths.
Runs in O(n + m) time.

This algorithm is based on \
https://cses.fi/problemset/task/2177
"""

```

```

def find_BCC(graph):
    d = 0
    depth = [0] * len(graph)

```

```

stack = list(range(len(graph)))
while stack:
    node = stack.pop()
    if node < 0:
        d -= 1
        elif not depth[node]:
            d = depth[node] = d + 1
            stack.append(~node)
            stack += graph[node]

graph2 = [[v for v in g if d + 2 > depth[v] != d - 1] \
for g,d in zip(graph, depth)]
return find_SCC(graph2)

```

graphs/floyd_warshall.py

```

def floyd_warshall(n, edges):
    dist = [[0 if i == j else float("inf") for i in \
range(n)] for j in range(n)]
    pred = [[None] * n for _ in range(n)]

    for u, v, d in edges:
        dist[u][v] = d
        pred[u][v] = u

    for k in range(n):
        for i in range(n):
            for j in range(n):
                if dist[i][k] + dist[k][j] < dist[i][j]:
                    dist[i][j] = dist[i][k] + dist[k][j]
                    pred[i][j] = pred[k][j]

    """Sanity Check
    for u, v, d in edges:
        if dist[u] + d < dist[v]:
            return None
    """

    return dist, pred

```

graphs/heavy_light.py

```

"""
Implementation of heavy-light decomposition of a tree.
This is intended to be combined with a segment tree in \
order to handle range queries on a tree in O(log^2 n).

It assigns labels to the nodes of a tree such that
* Every subtree is contained in a contiguous interval of \
labels.
* Any path between two nodes can be decomposed into O(log \
n) contiguous intervals of labels.

Features:
    HLD.LCA(u, v):
        Returns lowest common ancestor (LCA) of u and v. Runs \
in O(log n) time.

    HLD.node_query(u):
        Returns the label of u.

```

```

HLD.subtree_query(u):
    Returns interval [l,r] corresponding to subtree of u.

HLD.path_query(u,v):
    A generator that yields O(log n) intervals [l,r] \
    corresponding to
    the path from u to v. The intervals are given in \
    increasing order.

HLD.order:
    A list, where HLD.order[i] is the node with label i.

For how to use, see the following example:

https://codeforces.com/contest/343/problem/D
https://codeforces.com/contest/343/submission/327932478

'''py
HLD = HLD(graph)
SEG = SegmentTree(n)
for _ in range(q):
    c, v = [int(x) - 1 for x in input().split()]
    if c == 0:
        # Fill v's subtree
        l,r = HLD.subtree_query(v)
        SEG.update(l, r, 1)
    elif c == 1:
        # Empty v's ancestors (including v)
        for l,r in HLD.path_query(v, 0):
            SEG.update(l, r, 0)
    else:
        # Query v
        ind = HLD.node_query(v)
        print(SEG[ind])
'''

class HLD:
    def __init__(self, graph, root=0):
        n = len(graph)

        # Iterative BFS traversal
        P = [-1] * n
        P[root] = root
        BFS = [root]
        for u in BFS:
            for v in graph[u]:
                if P[v] == -1:
                    P[v] = u
                    BFS.append(v)

        # Compute subtree sizes
        size = [0] * n
        for u in BFS[::-1]:
            size[u] = 1 + sum(size[v] for v in graph[u])

        # Modify BFS to make heaviest child last
        for i in range(1, n - 1):
            if P[BFS[i]] == P[BFS[i + 1]] and size[BFS[i]] > \

```

```

size[BFS[i + 1]]:
    BFS[i],BFS[i + 1] = BFS[i + 1],BFS[i]

    # Compute postorder traversal from BFS
    post_trav = [0] * n
    for u in BFS[1:]:
        p = P[u]
        post_trav[u] = post_trav[p]
        post_trav[p] += size[u]

    # Invert post_trav
    self.order = order = [0] * n
    for i in range(n):
        order[post_trav[i]] = i

    # Remap P and size to post_trav order
    self.P = P = [post_trav[P[u]] for u in order]
    self.size = size = [size[u] for u in order]

    # Compute heavy path head
    self.HPH = HPH = [0] * n
    for i in range(n)[::-1]:
        if P[i] == i + 1:
            HPH[i] = HPH[i + 1]
        else:
            HPH[i] = i

    self.node_query = post_trav.__getitem__

    def _LCA(self, u, v):
        HPH = self.HPH
        P = self.P
        u,v = min(u,v),max(u,v)
        while HPH[u] < v:
            u = P[HPH[u]]
        return max(u,v)

    def _subtree_query(self, u):
        return u + 1 - self.size[u], u + 1

    def _path_query(self, u, v):
        HPH = self.HPH
        P = self.P
        u,v = min(u,v),max(u,v)
        while HPH[u] < v:
            yield u, HPH[u] + 1
            u = P[HPH[u]]
        while HPH[v] < u:
            yield v, HPH[v] + 1
            v = P[HPH[v]]
        yield min(u,v), max(u,v) + 1

    def LCA(self, u, v):
        return self.order[self._LCA(self.node_query(u), \
self.node_query(v))]
    def subtree_query(self, u):
        return self._subtree_query(self.node_query(u))
    def path_query(self, u, v):
        return self._path_query(self.node_query(u), \

```

```
self.node_query(v))
```

graphs/hopcroft_karp.py

```

"""
Produces a maximum cardinality matching of a bipartite graph

Example:

0---0
 \
  \
   \
1---1
 \ /
  /
 / \
2  2
 /
 /
 /
3

>>> n = 4
>>> m = 3
>>> graph = [[0, 1], [1, 2], [1], [2]]
>>> match1, match2 = hopcroft_karp(graph, n, m)
>>> match1
[0, 1, -1, 2]
>>> match2
[0, 1, 3]

Meaning
0---0
1---1
2  2
 /
 /
 /
3

"""

def hopcroft_karp(graph, n, m):
    """
    Maximum bipartite matching using Hopcroft-Karp \
    algorithm, running in O(|E| sqrt(|V|))
    """
    assert (n == len(graph))
    match1 = [-1] * n
    match2 = [-1] * m

    # Find a greedy match for possible speed up
    for node in range(n):
        for nei in graph[node]:
            if match2[nei] == -1:
                match1[node] = nei

```

```

        match2[nei] = node
        break
while 1:
    bfs = [node for node in range(n) if match1[node] == -1]
    depth = [-1] * n
    for node in bfs:
        depth[node] = 0

    for node in bfs:
        for nei in graph[node]:
            next_node = match2[nei]
            if next_node == -1:
                break
            if depth[next_node] == -1:
                depth[next_node] = depth[node] + 1
                bfs.append(next_node)
        else:
            continue
        break
    else:
        break

pointer = [len(c) for c in graph]
dfs = [node for node in range(n) if depth[node] == 0]
while dfs:
    node = dfs[-1]
    while pointer[node]:
        pointer[node] -= 1
        nei = graph[node][pointer[node]]
        next_node = match2[nei]
        if next_node == -1:
            # Augmenting path found
            while nei != -1:
                node = dfs.pop()
                match2[nei], match1[node], nei = node, nei, \
match1[node]
            break
        elif depth[node] + 1 == depth[next_node]:
            dfs.append(next_node)
            break
    else:
        dfs.pop()
return match1, match2

```

graphs/kruskal.py

```

class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))

    def find(self, a):
        acopy = a
        while a != self.parent[a]:
            a = self.parent[a]
        while acopy != a:
            self.parent[acopy], acopy = a, self.parent[acopy]
        return a

    def merge(self, a, b):
        self.parent[self.find(b)] = self.find(a)

```

```

def kruskal(n, U, V, W):
    union = UnionFind(n)
    cost, merge_cnt = 0, 0
    mst_u, mst_v = [], []
    order = sorted(range(len(W)), key=lambda x: W[x])
    for i in range(len(W)):
        u, v = U[order[i]], V[order[i]]
        find_u, find_v = union.find(u), union.find(v)
        if find_u != find_v:
            cost += W[order[i]]
            merge_cnt += 1
            union.parent[find_v] = find_u
            mst_u.append(u), mst_v.append(v)

    return cost, mst_u, mst_v, n == 1 + merge_cnt

```

graphs/lca.py

```

class RangeQuery:
    def __init__(self, data, func=min):
        self.func = func
        self._data = _data = [list(data)]
        i, n = 1, len(_data[0])
        while 2 * i <= n:
            prev = _data[-1]
            _data.append([func(prev[j], prev[j + i]) for j in \
range(n - 2 * i + 1)])
            i <= 1

    def query(self, begin, end):
        depth = (end - begin).bit_length() - 1
        return self.func(self._data[depth][begin], \
self._data[depth][end - (1 << depth)])

```

```

class LCA:
    def __init__(self, root, graph):
        self.time = [-1] * len(graph)
        self.path = [-1] * len(graph)
        P = [-1] * len(graph)
        t = -1
        dfs = [root]
        while dfs:
            node = dfs.pop()
            self.path[t] = P[node]
            self.time[node] = t = t + 1
            for nei in graph[node]:
                if self.time[nei] == -1:
                    P[nei] = node
                    dfs.append(nei)
            self.rm = RangeQuery(self.time[node] for node in \
self.path)

    def __call__(self, a, b):
        if a == b:
            return a
        a = self.time[a]
        b = self.time[b]

```

```

if a > b:
    a, b = b, a
return self.path[self.rm.query(a, b)]

```

graphs/maximum_matching.py

```

import itertools
from random import randint

_DEFAULT_PRIME = 1073750017

def maximum_matching(edges, mod=_DEFAULT_PRIME):
    """
    Returns the maximum cardinality matching of any simple \
graph (undirected, unweighted, no self-loops) \
Uses a randomized algorithm to compute the rank of the \
Tutte matrix \
The rank of the Tutte matrix is equal to twice the size \
of the maximum matching with high probability \
The probability for error is not more than n/mod

    Complexity: O(n ^ 3) worst case, O(n * |matching_size|) \
on average

    :param edges: a list of edges, assume nodes can be \
anything numbered from 0 to max number in edges
    :param mod: optional, a large random prime
    :return: the maximum cardinality matching of the graph
    """

    n = max(itertools.chain(*edges)) + 1
    matrix = _get_tutte_matrix(n, edges, mod)
    return _gauss(n, matrix, mod) // 2

def _get_tutte_matrix(n, edges, mod):
    matrix = [[0] * n for _ in range(n)]

    for u, v in edges:
        val = randint(1, mod - 1)
        matrix[u][v], matrix[v][u] = val, mod - val

    return matrix

def _gauss(n, matrix, mod):
    r = 0
    for j in range(n):
        k = r
        while k < n and not matrix[k][j]:
            k += 1

        if k == n:
            continue

        inv = pow(matrix[k][j], mod - 2, mod)
        for i in range(n):
            matrix[k][i] = inv * matrix[k][i] % mod
        matrix[k], matrix[r] = matrix[r], matrix[k]

```

```

for u in range(r + 1, n):
    # reducing indexing costs to gain performance boost \
for the next loop
    matrix_u, matrix_r = matrix[u], matrix[r]
    if matrix_u[j]:
        for v in range(j + 1, n):
            if matrix_r[v]:
                matrix_u[v] = (matrix_u[v] - matrix_r[v] * \
matrix_u[j]) % mod

        r += 1

return r

```

graphs/node_bcc.py

```

"""
REMARK: There are two definitions of biconnectedness of an \
undirected graph,
    either node-bcc or edge-bcc, corresponding to \
node_bcc.py and edge_bcc.py.

* In node-bcc, a "cut-vertex" (also called an articulation \
point) is a node where
    removing it would split up the graph into multiple \
connected components.

* In edge-bcc, a "bridge" is an edge where removing it \
would split up the graph
    into multiple connected components.

For example, this graph

  1  4  7  10
 / \ / \ / \ /
2  0  5  9
 \ / \ / \ /
  3  6  8

has three cut vertices (nodes 0,5,9) and one bridge (edge \
between 9 and 10).
The number of node-bcc = 4 and number of edge-bcc = 2.
Note that a node belongs to multiple node-bccs iff it is a \
cut-vertex.
"""

"""
Given an undirected graph, find_BCC returns a list of \
lists containing the nodes
of the different node-biconnected components.
Node-biconnected components are maximal subgraphs that \
cannot be disconnected
by removing a single node.
Note that a node belongs to multiple node-bccs iff it is \
an articulation point.
"""

def find_bcc(graph):
    n = len(graph)

```

```

BCC = []

P = [-1] * n
depth = [-1] * n
biconnect = [None] * n

preorder = []
stack = list(range(n))
while stack:
    node = stack.pop()
    if depth[node] >= 0:
        continue
    depth[node] = len(preorder)
    preorder.append(node)
    for nei in graph[node]:
        if depth[nei] == -1:
            P[nei] = node
            stack.append(nei)

for node in reversed(preorder):
    if P[node] != -1:
        depth[node] = min(depth[nei] for nei in graph[node])
        if depth[P[node]] == depth[node]:
            bicon = biconnect[node] = [P[node], node]
            BCC.append(bicon)

for node in preorder:
    if not graph[node]:
        BCC.append([node])
    elif P[node] != -1 and biconnect[node] is None:
        bicon = biconnect[node] = biconnect[P[node]]
        bicon.append(node)

return BCC

```

graphs/prim.py

```

def prim(n, adj):
    total_weight = 0
    selected, min_e = [False] * n, [[float("inf"), -1] for _ \
in range(n)]
    mst_edges = []

    min_e[0][0] = 0

    for _ in range(n):
        v = -1

        for j in range(n):
            if (not selected[j]) and ((v == -1) or (min_e[j][0] \
< min_e[v][0])):
                v = j

        if min_e[v][0] == float("inf"):
            return None, None

        selected[v] = True
        total_weight += min_e[v][0]

        if min_e[v][1] != -1:

```

```

mst_edges.append((v, min_e[v][1]))

for to in range(n):
    if adj[v][to] < min_e[to][0]:
        min_e[to] = [adj[v][to], v]

return mst_edges, total_weight

```

graphs/rerooter.py

```

# for usage see https://codeforces.com/blog/entry/124286

def rerooter(graph, default, combine, finalize=lambda \
nodeDP, node, eind: nodeDP):
    n = len(graph)
    rootDP = [0] * n
    forwardDP = [None] * n
    reverseDP = [None] * n

    def exclusive(A, zero, combine, node):
        n = len(A)
        exclusiveA = [zero] * n

        for bit in range(n.bit_length()[::-1]):
            for i in range(n)[::-1]:
                exclusiveA[i] = exclusiveA[i // 2]
            for i in range(n & (-1 if bit else -2)):
                ind = (i >> bit) ^ 1
                exclusiveA[ind] = combine(exclusiveA[ind], A[i], \
node, i)
            return exclusiveA

    DP = [0] * n
    bfs = [0]
    P = [0] * n
    for node in bfs:
        for nei in graph[node]:
            if P[node] != nei:
                P[nei] = node
                bfs.append(nei)

    for node in reversed(bfs):
        nodeDP = default[node]
        for eind, nei in enumerate(graph[node]):
            if P[node] != nei:
                nodeDP = combine(nodeDP, DP[nei], node, eind)
        DP[node] = finalize(nodeDP, node, \
graph[node].index(P[node]) if node else -1)

    for node in bfs:
        DP[P[node]] = DP[node]
        forwardDP[node] = [DP[nei] for nei in graph[node]]
        rerootDP = exclusive(forwardDP[node], default[node], \
combine, node)
        reverseDP[node] = [
            finalize(nodeDP, node, eind) for eind, nodeDP in \
enumerate(rerootDP)
        ]
        rootDP[node] = finalize(
            (

```

```

        combine(rerootDP[0], forwardDP[node][0], node, 0)
        if n > 1
        else default[node]
    ),
    node,
    -1,
)
for nei, dp in zip(graph[node], reverseDP[node]):
    DP[nei] = dp
return rootDP, forwardDP, reverseDP

```

graphs/scc.py

"""
Given a directed graph, find_SCC returns a list of lists \
containing
the strongly connected components in topological order.

Note that this implementation can be also be used to check \
if a directed graph is a
DAG, and in that case it can be used to find the \
topological ordering of the nodes.
"""

```

def find_SCC(graph):
    SCC, S, P = [], [], []
    depth = [0] * len(graph)

    stack = list(range(len(graph)))
    while stack:
        node = stack.pop()
        if node < 0:
            d = depth[~node] - 1
            if P[~1] > d:
                SCC.append(S[d:])
                del S[d:], P[~1]
                for node in SCC[~1]:
                    depth[node] = -1
            elif depth[node] > 0:
                while P[~1] > depth[node]:
                    P.pop()
            elif depth[node] == 0:
                S.append(node)
                P.append(len(S))
                depth[node] = len(S)
                stack.append(~node)
                stack += graph[node]
    return SCC[::-1]

```

graphs/toposort.py

```

# from heapq import heappop, heappush

def toposort(graph):
    res, found = [], [0] * len(graph)
    stack = list(range(len(graph)))
    while stack:
        node = stack.pop()
        if node < 0:

```

```

            res.append(~node)
        elif not found[node]:
            found[node] = 1
            stack.append(~node)
            stack += graph[node]

    # cycle check
    for node in res:
        if any(found[nei] for nei in graph[node]):
            return None
        found[node] = 0

    return res[::-1]

```

```

def kahn(graph):
    n = len(graph)

    indeg, idx = [0] * n, [0] * n
    for i in range(n):
        for e in graph[i]:
            indeg[e] += 1

    q, res = [], []
    for i in range(n):
        if indeg[i] == 0:
            q.append(i) # heappush(q, -i)

    nr = 0
    while q:
        res.append(q.pop()) # res.append(-heappop(q))
        idx[res[-1]], nr = nr, nr + 1
        for e in graph[res[-1]]:
            indeg[e] -= 1
            if indeg[e] == 0:
                q.append(e) # heappush(q, -e)

    return res, idx, nr == n

```

linear_algebra/matrix.py

```

from copy import deepcopy

transpose = lambda mat: [list(col) for col in zip(*mat)]

minor = lambda mat, i, j: [row[:j] + row[j + 1:] for row \
    in (mat[:i] + mat[i + 1:])]

mat_add = lambda *mat: [[sum(elements) for elements in \
    zip(*row)] for row in zip(*mat)]

mat_sub = lambda A, B: [[i - j for i, j in zip(*row)] for \
    row in zip(A, B)]

mat_mul = lambda A, B: [[sum(i * j for i, j in zip(row, \
    col)) for col in zip(*B)] for row in A]

vec_mul = lambda mat, vec: [sum(a * b for a, b in zip(row, \
    vec)) for row in mat]

```

```

def eye(m):
    """returns an identity matrix of order m"""
    identity = [[0] * m for _ in range(m)]
    for i, row in enumerate(identity):
        row[i] = 1
    return identity

def mat_pow(mat, power):
    """returns mat**power"""
    if power < 0:
        return mat_pow(mat_inv(mat), -power)

    result = eye(len(mat))
    if power == 0:
        return result

    while power > 1:
        if power & 1 == 1:
            result = mat_mul(result, mat)
        mat = mat_mul(mat, mat)
        power >>= 1
    return mat_mul(result, mat)

def mat_inv(A):
    B = deepcopy(A)
    n = len(A)
    col = list(range(n))

    tmp = [[0] * n for _ in range(n)]
    for i in range(n):
        tmp[i][i] = 1

    for i in range(n):
        r = col[i]
        for j in range(i, n):
            for k in range(i, n):
                if abs(B[j][k]) > abs(B[r][c]):
                    r, c = j, k
            if B[r][c] == 0:
                return B

        B[i], B[r] = B[r], B[i]
        tmp[i], tmp[r] = tmp[r], tmp[i]
        for j in range(n):
            B[j][i], B[j][c] = B[j][c], B[j][i]
            tmp[j][i], tmp[j][c] = tmp[j][c], tmp[j][i]
        col[i], col[c] = col[c], col[i]
        v = B[i][i]
        for j in range(i + 1, n):
            f = B[j][i] / v
            B[j][i] = 0
            for k in range(i + 1, n):
                B[j][k] -= f * B[i][k]
            for k in range(n):
                tmp[j][k] -= f * tmp[i][k]

```



```

L = i + 1 - L
B, b, m = T[:], d, 0

return [-C[i] % MOD for i in range(1, L + 1)]

def linear_rec(S, tr, k, MOD=10**9 + 7):
    n = len(S)

    def combine(a, b):
        res = [0] * (2 * n + 1)
        for i in range(n + 1):
            for j in range(n + 1):
                res[i + j] = (res[i + j] + a[i] * b[j]) % MOD

        for i in range(2 * n, n, -1):
            for j in range(n):
                res[i - 1 - j] = (res[i - 1 - j] + res[i] * tr[j]) \
% MOD
        return res[:n + 1]

    pol, e = [0] * (n + 1), [0] * (n + 1)
    pol[0], e[1] = 1, 1

    k += 1
    while k:
        if k & 1:
            pol = combine(pol, e)
            e = combine(e, e)
            k >>= 1

    res = 0
    for i in range(n):
        res = (res + pol[i + 1] * S[i]) % MOD
    return res

```

numerical/integrate.py

```

def quad(func, a, b, n=1000):
    h = (b - a) / 2 / n
    v = func(a) + func(b) + sum(func(a + i * h) * (4 if i & \
1 else 2) for i in range(1, 2 * n))
    return v * h / 3

def simpson(func, a, b):
    c = (a + b) / 2
    return (func(a) + 4 * func(c) + func(b)) * (b - a) / 6

def rec(func, a, b, eps, S):
    c = (a + b) / 2
    S1, S2 = simpson(func, a, c), simpson(func, c, b)
    if (abs(S1 + S2 - S) <= 15 * eps) or (b - a < 1e-6):
        return S1 + S2 + (S1 + S2 - S) / 15
    return rec(func, a, c, eps / 2, S1) + rec(func, c, b, \
eps / 2, S2)

def fast_quad(func, a, b, eps=1e-6):

```

```

return rec(func, a, b, eps, simpson(func, a, b))

```

numerical/search.py

```

def binary_search(func, lo, hi, abs_prec=1e-7):
    """ Locate the first value x s.t. func(x) = True within \
[lo, hi] """
    while abs(hi - lo) > abs_prec:
        mi = lo + (hi - lo) / 2
        if func(mi):
            hi = mi
        else:
            lo = mi
    return (lo + hi) / 2

def ternary_search(func, lo, hi, abs_prec=1e-7):
    """ Find maximum of unimodal function func() within [lo, \
hi] """
    while abs(hi - lo) > abs_prec:
        lo_third = lo + (hi - lo) / 3
        hi_third = hi - (hi - lo) / 3

        if func(lo_third) < func(hi_third):
            lo = lo_third
        else:
            hi = hi_third
    return (lo + hi) / 2

def discrete_binary_search(func, lo, hi):
    """ Locate the first value x s.t. func(x) = True within \
[lo, hi] """
    while lo < hi:
        mi = lo + (hi - lo) // 2
        if func(mi):
            hi = mi
        else:
            lo = mi + 1
    return lo

def discrete_ternary_search(func, lo, hi):
    """ Find the first maximum of unimodal function func() \
within [lo, hi] """
    while lo <= hi:
        lo_third = lo + (hi - lo) // 3
        hi_third = lo + (hi - lo) // 3 + (1 if 0 < hi - lo < 3 \
else (hi - lo) // 3)

        if func(lo_third) < func(hi_third):
            lo = lo_third + 1
        else:
            hi = hi_third - 1
    return lo

```

```

def fractional_binary_search(func, lo=(0, 1), hi=(1, 0), \
limit=1000000):
    if func(lo):
        return lo

    flag = True
    A, B = 1, 1
    while A or B:
        adv, step = 0, 1

        si = 0
        while step:
            adv += step
            mid = (lo[0] * adv + hi[0], lo[1] * adv + hi[1])
            if abs(mid[0]) > limit or mid[1] > limit or flag != \
func(mid):
                adv -= step
                si = 2
            step += step
            step >>= si

        flag = not flag
        lo, hi = (hi[0] + lo[0] * adv, hi[1] + lo[1] * adv), lo
        A, B = B, adv

    return hi if flag else lo

```

```

def golden_section_search(a, b, func, abs_prec=1e-7):
    r = ((5**0.5) - 1) / 2

    x1 = b - r * (b - a)
    f1 = func(b - r * (b - a))

    x2 = a + r * (b - a)
    f2 = func(a + r * (b - a))

    while b - a > abs_prec:
        if f1 < f2:
            b, x2, f2 = x2, x1, f1
            x1 = b - r * (b - a)
            f1 = func(b - r * (b - a))
        else:
            a, x1, f1 = x1, x2, f2
            x2 = a + r * (b - a)
            f2 = func(a + r * (b - a))

    return a

```

strings/LPSubstr.py

```

def LPSubstr(s):
    n = len(s)
    p = [[0] * (n + 1), [0] * n]

    for z, p_z in enumerate(p):
        left, right = 0, 0
        for i in range(n):
            t = right - i + 1 - z

```

```

    if i < right:
        p_z[i] = min(t, p_z[left + t])
        L, R = i - p_z[i], i + p_z[i] - 1 + z
        while (L >= 1) and (R + 1 < n) and (s[L - 1] == s[R \
+ 1]):
            p_z[i] += 1
            L -= 1
            R += 1
        if R > right:
            left, right = L, R

i1, x1 = max(enumerate(p[0]), key=lambda x: x[1])
i2, x2 = max(enumerate(p[1]), key=lambda x: x[1])

return s[i1 - x1:i1 + x1], s[i2 - x2:i2 + x2 + 1]

```

strings/ hashing.py

```

import random

HMOD = 2147483647
HBASE1 = random.randrange(HMOD)
HBASE2 = random.randrange(HMOD)

class Hashing:
    def __init__(self, s, mod=HMOD, base1=HBASE1, \
base2=HBASE2):
        self.mod, self.base1, self.base2 = mod, base1, base2
        self._len = _len = len(s)
        f_hash, f_pow = [0] * (_len + 1), [1] * (_len + 1)
        s_hash, s_pow = f_hash[:], f_pow[:]
        for i in range(_len):
            f_hash[i + 1] = (base1 * f_hash[i] + s[i]) % mod
            s_hash[i + 1] = (base2 * s_hash[i] + s[i]) % mod
            f_pow[i + 1] = base1 * f_pow[i] % mod
            s_pow[i + 1] = base2 * s_pow[i] % mod
        self.f_hash, self.f_pow = f_hash, f_pow
        self.s_hash, self.s_pow = s_hash, s_pow

    def hashed(self, start, stop):
        return (
            (self.f_hash[stop] - self.f_pow[stop - start] * \
self.f_hash[start]) % self.mod,
            (self.s_hash[stop] - self.s_pow[stop - start] * \
self.s_hash[start]) % self.mod,
        )

    def get_hashes(self, length):
        return (
            [(self.f_hash[i + length] - self.f_pow[length] * \
self.f_hash[i]) % self.mod for i in range(self._len - \
length + 1)],
            [(self.s_hash[i + length] - self.s_pow[length] * \
self.s_hash[i]) % self.mod for i in range(self._len - \
length + 1)],
        )

```

strings/kmp.py

```

def partial(s):
    g, pi = 0, [0] * len(s)
    for i in range(1, len(s)):
        while g and (s[g] != s[i]):
            g = pi[g - 1]
        pi[i] = g = g + (s[g] == s[i])

    return pi

```

```

def match(s, pat):
    pi = partial(pat)

```

```

    g, idx = 0, []
    for i in range(len(s)):
        while g and pat[g] != s[i]:
            g = pi[g - 1]
        g += pat[g] == s[i]
        if g == len(pi):
            idx.append(i + 1 - g)
            g = pi[g - 1]

    return idx

```

```

def string_find(s, pat):
    pi = partial(pat)

```

```

    g = 0
    for i in range(len(s)):
        while g and pat[g] != s[i]:
            g = pi[g - 1]
        g += pat[g] == s[i]
        if g == len(pi):
            return True

```

```

    return False

```

strings/lcs.py

```

def lcs(a, b):
    lengths = [[0] * (len(b) + 1) for _ in range(len(a) + 1)]

    for i, x in enumerate(a):
        for j, y in enumerate(b):
            if x == y:
                lengths[i + 1][j + 1] = lengths[i][j] + 1
            else:
                lengths[i + 1][j + 1] = max(lengths[i + 1][j], \
lengths[i][j + 1])

    result = []
    x, y = len(a), len(b)
    while x != 0 and y != 0:
        if lengths[x][y] == lengths[x - 1][y]:
            x -= 1
        elif lengths[x][y] == lengths[x][y - 1]:
            y -= 1
        else:

```

```

        result.append(a[x - 1])
        x -= 1
        y -= 1

    return result[::-1]

```

```

def lps(s):
    return lcs(s, s[::-1])

```

strings/min_rotation.py

```

def least_rotation(s):
    a, n = 0, len(s)
    s = s + s

    b = 1
    while b < n:
        for i in range(b - a):
            if s[a + i] > s[b + i]:
                a = b
                b += 1
                break
            if s[a + i] < s[b + i]:
                b += i + 1
                break
        else:
            b += b - a
    return s[a:a + n]

```

strings/suffix_array.py

```

"""
Calculates the suffix array and LCP array in O(n) time

Example:
>>> S = 'cabbage'
>>> SA = SAIS([ord(c) for c in S])
>>> LCP = KASAI(S, SA)
>>> SA
[1, 4, 3, 2, 0, 6, 5]
>>> LCP
[1, 0, 1, 0, 0, 0]

"""

def SAIS(A):
    """
    Calculates suffix array in O(len(A) + max(A))
    Input:
    Int list A with A[i] >= 0 for all i
    """
    n = len(A)
    buckets = [0] * (max(A) + 2)
    for a in A:
        buckets[a + 1] += 1
    for b in range(1, len(buckets)):
        buckets[b] += buckets[b - 1]
    isL = [1] * n

```

```

    for i in reversed(range(n - 1)):
        isL[i] = +(A[i] > A[i + 1]) if A[i] != A[i + 1] else \
isL[i + 1]

def induced_sort(LMS):
    SA = [-1] * (n)
    SA.append(n)
    endpoint = buckets[1:]
    for j in reversed(LMS):
        endpoint[A[j]] -= 1
        SA[endpoint[A[j]]] = j
    startpoint = buckets[:-1]
    for i in range(-1, n):
        j = SA[i] - 1
        if j >= 0 and isL[j]:
            SA[startpoint[A[j]]] = j
            startpoint[A[j]] += 1
    SA.pop()
    endpoint = buckets[1:]
    for i in reversed(range(n)):
        j = SA[i] - 1
        if j >= 0 and not isL[j]:
            endpoint[A[j]] -= 1
            SA[endpoint[A[j]]] = j
    return SA

isLMS = [(i and isL[i - 1] and not isL[i]) for i in \
range(n)]
isLMS.append(1)
LMS = [i for i in range(n) if isLMS[i]]
if len(LMS) > 1:
    SA = induced_sort(LMS)
    LMS2 = [i for i in SA if isLMS[i]]
    prev = -1
    j = 0
    for i in LMS2:
        i1 = prev
        i2 = i
        while prev >= 0 and A[i1] == A[i2]:
            i1 += 1
            i2 += 1
        if isLMS[i1] or isLMS[i2]:
            j -= isLMS[i1] and isLMS[i2]
            break
        j += 1
        prev = i
        SA[i] = j
    LMS = [LMS[i] for i in SAIS([SA[i] for i in LMS])]
    return induced_sort(LMS)

def KASAI(A, SA):
    """
    Calculates LCP array in O(n) time
    Input:
    String A and its suffix array SA
    """
    n = len(A)
    rank = [0] * n

```

```

    for i in range(n):
        rank[SA[i]] = i
    LCP = [0] * (n - 1)
    k = 0
    for i in range(n):
        SAind = rank[i]
        if SAind == n - 1:
            continue
        j = SA[SAind + 1]
        while i + k < n and A[i + k] == A[j + k]:
            k += 1
        LCP[SAind] = k
        k -= k > 0
    return LCP

```

strings/z_algorithm.py

```

def z_function(S):
    """
    Z Algorithm in O(n)
    :param S: text string to process
    :return: the Z array, where Z[i] = length of the longest \
common prefix of S[i:] and S
    """
    n = len(S)
    Z = [0] * n
    l = r = 0

    for i in range(1, n):
        z = Z[i - l]
        if i + z >= r:
            z = max(r - i, 0)
            while i + z < n and S[z] == S[i + z]:
                z += 1

            l, r = i, i + z

        Z[i] = z

    Z[0] = n
    return Z

```